Rapport de Securisation et Remediation

Projet final – Cyber Challenge

Blue Team

Auteur	Arthur LAUZANNE
Filiere	Cyber
Binome	Romain LIGNERES - Red team
Ecole	ECE Paris
Annee	2025-2026
Date	10 avril 2026

Table des matières

1 Introduction **3**

2 Perimetre et Strategie **3**

3 Vulnerabilite 1 – Injection SQL **4**
3.1 Localisation 4
3.2 Constat Red Team 4
3.3 Remediation appliquee 4
3.4 Preuve de correction 4

4 Vulnerabilite 2 – XSS Reflechi **5**
4.1 Localisation 5
4.2 Constat Red Team 5
4.3 Remediation appliquee 5
4.4 Preuve de correction 5

5 Vulnerabilite 3 – XSS Stocke **6**
5.1 Localisation 6
5.2 Constat Red Team 6
5.3 Remediation appliquee 6
5.4 Preuve de correction 6

6 Vulnerabilite 4 – IDOR **7**
6.1 Localisation 7
6.2 Constat Red Team 7
6.3 Remediation appliquee 7
6.4 Preuve de correction 7

7 Vulnerabilite 5 – Path Traversal **8**
7.1 Localisation 8
7.2 Constat Red Team 8
7.3 Remediation appliquee 8
7.4 Preuve de correction 8

8 Vulnerabilite 6 – Broken Access Control **9**
8.1 Localisation 9
8.2 Constat Red Team 9
8.3 Remediation appliquee 9
8.4 Preuve de correction 9

9 Vulnerabilite 7 – Information Disclosure **10**
9.1 Localisation 10
9.2 Constat Red Team 10
9.3 Remediation appliquee 10

9.4 Preuve de correction 10

10 Vulnerabilite 8 – Open Redirect **11**
10.1 Localisation 11
10.2 Constat Red Team 11
10.3 Remediation appliquee 11
10.4 Preuve de correction 11

11 Vulnerabilite 9 – OS Command Injection **13**
11.1 Localisation 13
11.2 Constat Red Team 13
11.3 Remediation appliquee 13
11.4 Preuve de correction 13

12 Conclusion **14**

Introduction

Ce rapport documente la phase de remediation menee par la Blue Team suite a l'audit de securite (pentest) realise par la Red Team sur notre facade Moodle simplifiee.

L'objectif de cette phase etait d'analyser le rapport d'intrusion, de comprendre les vecteurs d'attaque utilises, et de concevoir des correctifs robustes pour neutraliser definitivement les 9 vulnerabilites identifiees par la Red Team, tout en maintenant les fonctionnalites de l'application intactes.

Perimetre et Strategie

Element	Detail
Application	Facade Moodle simplifiee (Blue Team)
URL	`http://localhost:3000`
Technologies	Node.js, Express.js, express-session, SQLite

Notre strategie de defense en profondeur s'est articulee autour de trois axes majeurs :

1. **La validation et la sanetisation** (protection contre les Injections et XSS).
2. **La gestion de session securisee** cote serveur (protection IDOR et Controle d'acces).
3. **L'application de Listes Blanches** (protection Path Traversal et Injection de commandes).

Vulnerabilite 1 – Injection SQL

Localisation

Route `POST /` – Traitement du formulaire de connexion.

Constat Red Team

Les entrees utilisateur etaient concatenees directement dans la requete SQL, permettant un contournement total de l'authentification avec le payload `' OR '1'='1`.

Remediation appliquee

Nous avons remplace la concatenation par des **requetes preparees**. Le moteur SQLite traite desormais les entrees strictement comme des chaines de caracteres.

```
const query = `SELECT * FROM users WHERE username = ? AND
   password = ?`;
db.get(query, [username, password], (err, row) => { ... });
```

Preuve de correction

L'injection SQL est desormais traitee comme une simple tentative de connexion erronee.

Figure 1 – Echec de l'injection SQL - Identifiants incorrects

Vulnerabilite 2 – XSS Reflechi

Localisation

Route `GET /calendar` – Parametre de recherche.

Constat Red Team

Le texte recherche etait reflete sans filtrage dans le code HTML, permettant l'execution de scripts JS.

Remediation appliquee

Creation et application systematique d'une fonction `escapeHTML()` qui convertit les caracteres speciaux en entites inoffensives avant l'affichage.

```
const safeRecherche = escapeHTML(motRecherche);
const texteAffiche = `<span style="color: red;">Resultats
   pour : ${safeRecherche}</span>`;
```

Preuve de correction

La balise script est affichee en texte brut et n'est plus interpretee par le navigateur.

FIGURE 2 – Neutralisation du payload XSS reflechi

Vulnerabilite 3 – XSS Stocke

Localisation

Route `POST /add-comment` et `GET /calendar`.

Constat Red Team

Les commentaires inseres en base n'etaient ni filtres a l'entree ni a la sortie, permettant une execution de code permanente.

Remediation appliquee

Utilisation d'une requete preparee pour l'insertion en base, puis application de la fonction `escapeHTML()` lors du rendu de la liste des commentaires.

```
let commentairesHtml = rows.map(r => `<p>- ${escapeHTML(r.
    text)}</p>`).join('');
```

Preuve de correction

Les balises HTML saisies par les utilisateurs sont affichees litteralement.

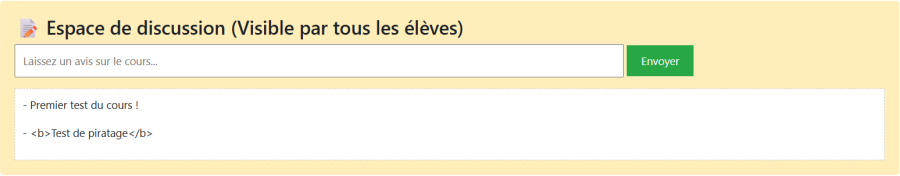

FIGURE 3 – Neutralisation du XSS Stocke dans les commentaires

Vulnerabilite 4 – IDOR

Localisation

Route `GET /calendar?user=student`

Constat Red Team

Le role de l'utilisateur etait determine par un parametre modifiable dans l'URL (`?user=admin`).

Remediation appliquee

Mise en place du module `express-session`. L'identite et le role sont desormais stockes cote serveur apres verification des identifiants.

```
// Assignation au login : req.session.role = row.role;
const role = req.session.role; // Lecture securisee
```

Preuve de correction

Meme en forcant `?user=admin`, la session etudiant est maintenue.

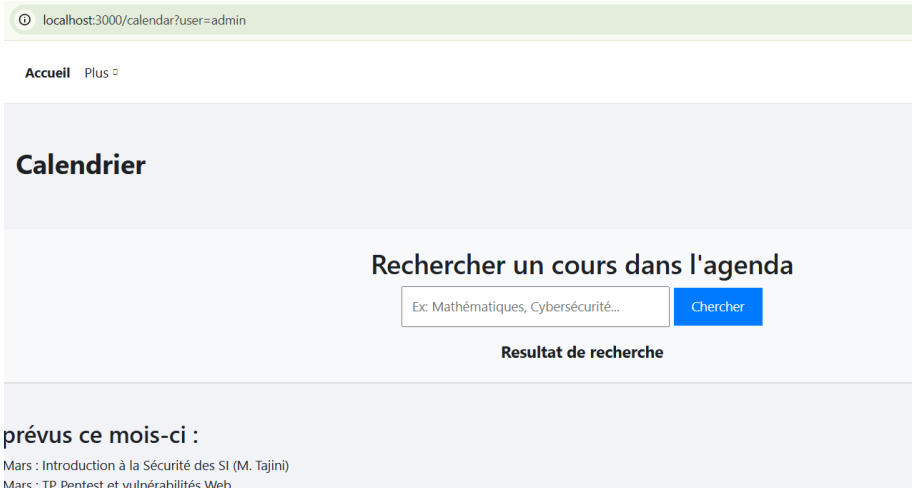

FIGURE 4 – Echec de l'elevation de privileges via IDOR

Vulnerabilite 5 – Path Traversal

Localisation

Route `GET /download`

Constat Red Team

Le parametre `file` permettait de telecharger n'importe quel fichier du serveur.

Remediation appliquee

Abandon du filtrage simple au profit d'une approche par **Liste Blanche (Whitelist)**. Seuls les fichiers explicitement declares dans le code peuvent etre servis.

```
const fichiersAutorises = ['syllabus.pdf'];
if (!fichiersAutorises.includes(file)) {
    return res.status(403).send("Acces Interdit.");
}
```

Preuve de correction

Le serveur rejette explicitement toute demande de fichier non inscrite sur la liste blanche.

Accès Interdit. Tentative de téléchargement non autorisée bloquée.

FIGURE 5 – Blocage de la tentative de telechargement du code source

Vulnerabilite 6 – Broken Access Control

Localisation

Route `GET /admin`

Constat Red Team

Le panneau d'administration critique etait accessible publiquement par simple connaissance de l'URL.

Remediation appliquee

Ajout d'un controle de session strict. Le serveur verifie desormais que la requete provient d'un utilisateur authentifie possedant explicitement le role "admin".

```
if (req.session.role !== 'admin') {
    return res.status(403).send("Acces Interdit. Vous n'
       etes pas professeur.");
}
```

Preuve de correction

Une tentative d'acces non autorisee renvoie une erreur 403.

Accès Interdit. Vous n'êtes pas professeur.

FIGURE 6 – Interdiction d'acces a la zone d'administration

Vulnerabilite 7 – Information Disclosure

Localisation

Route `GET /debug/db`

Constat Red Team

Une route de developpement obsolete permettait de dumper l'integralite de la base de donnees sans authentification.

Remediation appliquee

Suppression definitive de la route de debug dans le code source de production.

Preuve de correction

Le serveur ne reconnait plus cette route et renvoie une erreur standard.

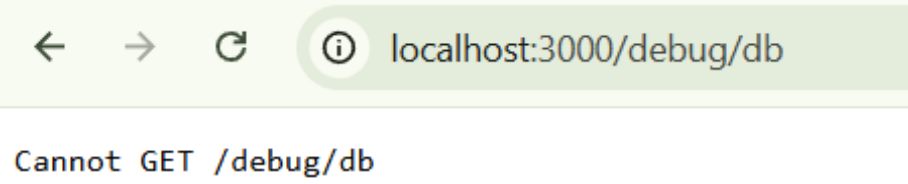

FIGURE 7 – Route de debug rendue inaccessible

Vulnerabilite 8 – Open Redirect

Localisation

Route `POST /` (parametre de redirection `next`)

Constat Red Team

L'application redirigeait aveuglement l'utilisateur vers l'URL fournie dans le parametre `next`.

Remediation appliquee

Validation stricte de l'URL cible. Le serveur exige desormais que l'URL commence par le caractere `/` (chemin relatif local).

```
let redirectUrl = req.query.next;
if (!redirectUrl || !redirectUrl.startsWith('/')) {
    redirectUrl = '/calendar';
}
```

Preuve de correction

L'injection d'un domaine complet est ignoree. La premiere image montre la tentative avec le parametre piégé `next`, et la seconde montre que l'utilisateur est bien redirige en toute securite vers `/calendar`.

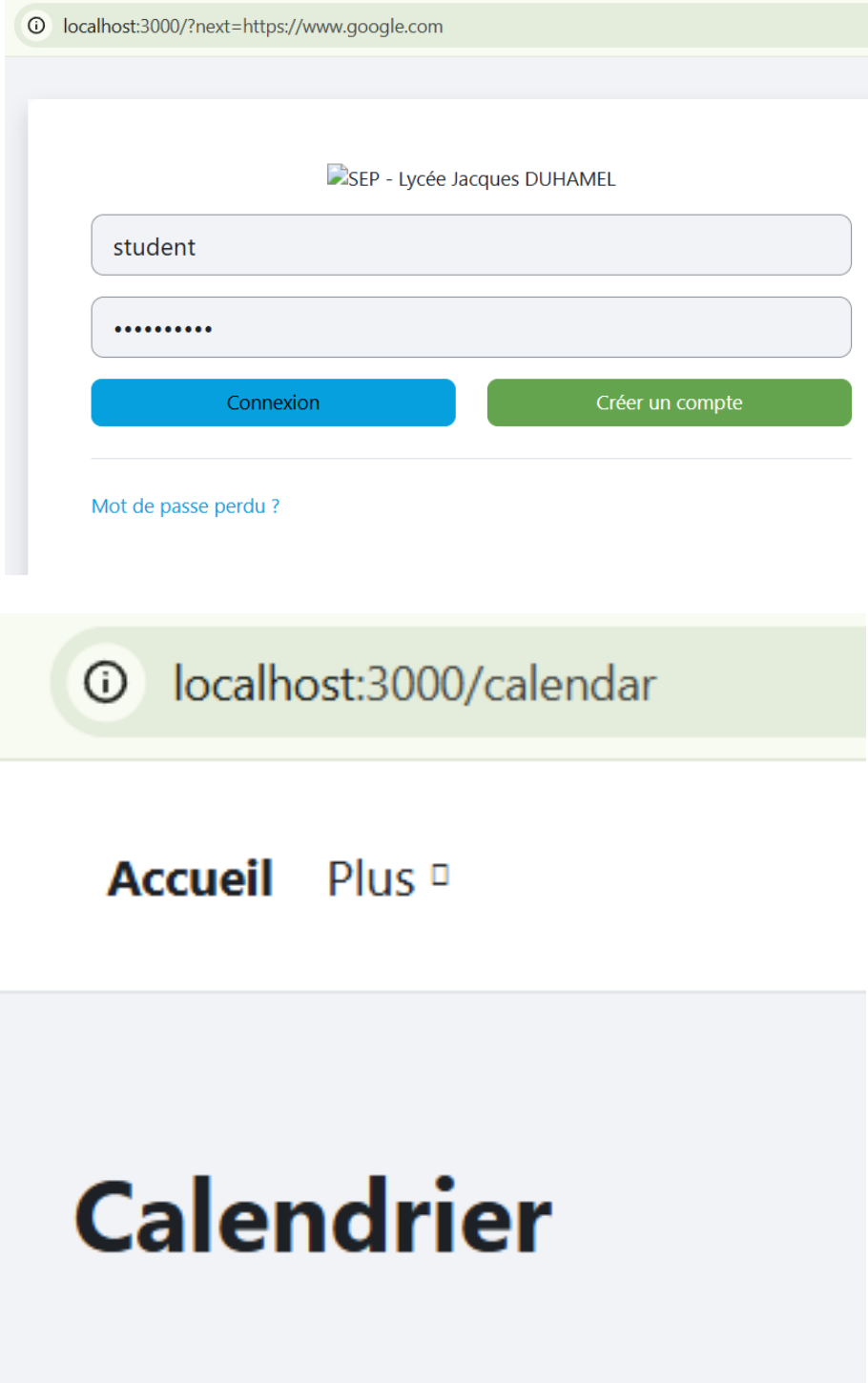

FIGURE 8 – Tentative d'Open Redirect (haut) et redirection forcee en local (bas)

Vulnerabilite 9 – OS Command Injection

Localisation

Route `POST /ping`

Constat Red Team

L'outil de diagnostic executait l'entree utilisateur directement dans le shell systeme, permettant d'enchainer des commandes malveillantes via le caractere `&`.

Remediation appliquee

Mise en place d'une **Expression Reguliere (Regex)** stricte agissant comme une liste blanche de caracteres.

```
if (!/^[a-zA-Z0-9.\-]+$/.test(ip)) {
    return res.send("Format IP/Domaine invalide.");
}
```

Preuve de correction

Toute tentative d'injection de caracteres speciaux est bloquee en amont. La premiere image montre la saisie du payload malveillant, et la seconde confirme son interception par le serveur.

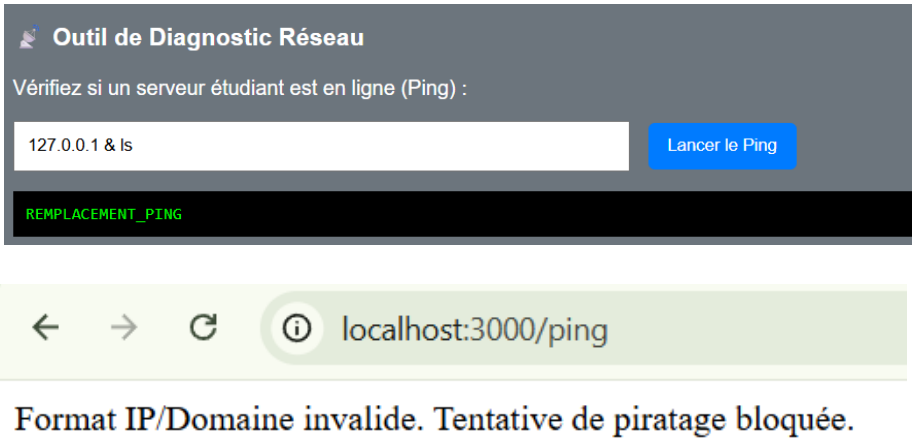

FIGURE 9 – Saisie du payload malveillant (haut) et blocage par la Regex (bas)

Conclusion

La campagne de remediation a permis de neutraliser avec succes 100% des vulnerabilites (9/9) documentees par la Red Team lors de son audit.

Ce processus a mis en evidence le fait que la securite ne s'ajoute pas a posteriori, mais doit s'integrer des la conception (Secure by Design). L'utilisation de sessions securisees et de listes blanches nous a permis de passer d'une application de type "passoire" a une architecture resiliente face aux attaques ciblees.

Arthur LAUZANNE – Blue Team – ECE Paris 2025-2026